

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ

УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи №4

«Рефакторинг та Code review»

з дисципліни «Основи програмної інженерії»

Виконав:

ст. гр. ПЗПІ-25-3

Крутько Д. С.

Прийняла:

Гребенюк В.О.

Харків 2026

1. Тема та мета лабораторної роботи

Тема: Рефакторинг коду та проведення Code Review за індустріальними стандартами.

Мета роботи:

Набуття практичних навичок проведення Code Review з використанням GitHub Pull Requests/Issues, ідентифікації типових проблем якості коду (code smells) та застосування рефакторинг-операцій з верифікацією збереження поведінки через регресійне тестування.

Проект: HookCraft – AI-сервіс для створення «гачків» для контенту.

Мова: Python 3.11. Файл, що рецензується: hook_generator.py.

Команда: Гиренко Владислава (автор коду) та Доброскокіна Анастасія (рецензент).

Дата виконання: 09.06.2026.

2. Результати Code Review одnogрупника

У ході Code Review файлу hook_generator.py (автор – Доброскокіна Анастасія) виявлено 7 проблем якості коду. Кожна проблема задокументована у вигляді GitHub Issue з посиланням на конкретні рядки, категорією code smell та рекомендацією щодо виправлення.

Таблиця 1 – Таблиця виявлених проблем

№	Рядок коду	Проблема	Категорія	Рекомендація
1	Рядок 2: VALID_STYLE S = [...]	VALID_STYLES є публічним list — зовнішній код може додавати або видаляти елементи через HookGenerator.VALID_STYLE S.append(...). Інваріант класу порушено.	Mutable State / Encapsulation Violation	Замінити на tuple або frozenset: VALID_STYLES = ("інтригуючий", ...) — незмінна колекція
2	Рядок 12: [s.lower() for s in self.VALID_STYLES]	При кожному виклику validate_style створюється новий список через list comprehension. За умови масових викликів (BulkGenerate) це зайве навантаження на GC.	Performance Smell / Repeated Allocation	Обчислити нормалізований набір один раз: _VALID_STYLES_LOWER = frozenset(s.lower() for s in VALID_STYLES) і порівнювати з ним
3	Рядки 22–26: if style == "інтригуючий"	validate_style робить style.strip().lower() перед порівнянням, але generate_hook порівнює style напряду без strip/lower. Якщо передати "інтригуючий", validate_style поверне True, але жоден if не спрацює — повернеться дефолтний хук.	Inconsistent Comparison / Logic Bug	Нормалізувати style на початку generate_hook: style = style.strip().lower(), або завжди порівнювати через casefold()
4	Рядок 19: topic_lower = topic.lower()	Замість явного StringComparison використовується magic workaround — окрема змінна topic_lower. Це приховує намір порівняння без урахування регістру та засмічує score.	Magic Workaround / Code Smell	Використати str.__contains__ з casefold: "рецепт" in topic.casefold() — прибирає проміжну змінну

5	Рядки 19–30:if/elif/else в generate_hook	Метод містить 3 рівні вкладених умов (тема → стиль → else). Цикломатична складність ≈ 8. Метод виконує і валідацію, і роутинг, і генерацію тексту — порушення SRP.	Deep Nesting / High Cyclomatic Complexity / SRP Violation	Виокремити _get_food_hook(topic, style) та _get_marketing_hook(style) як приватні методи; у generate_hook залишити лише роутинг
6	Рядок 4:self.history = []	Список history необмежено зростає. При довготривалій роботі сервісу або масових генераціях це призведе до витоку пам'яті.	Unbounded Memory Growth	Обмежити розмір: використати collections.deque(maxlen=1000) або додати метод clear_history()
7	Рядки 14–16:if not topic or not isinstance(topic, str)	Порядок перевірок у generate_hook некоректний: not topic спрацює для порожнього рядка, але isinstance перевіряється після. Якщо topic=0 (int), not topic = True і виняток кинеться з повідомленням про рядок, хоча реальна причина — неправильний тип.	Incorrect Validation Order / Misleading Error	Змінити порядок: спочатку isinstance, потім перевірка на порожнечу. Або розділити на два окремі if з різними повідомленнями

3. Посилання на PR з коментарями Code Review

Посилання на GitHub Pull Request, де задокументовано результати Code Review:

<https://github.com/vladagirenko/hookcraft/pull/3#pullrequestreview-4461781968>

У PR залишено 5 inline-коментарів безпосередньо до рядків коду з описом проблеми, категорії code smell та рекомендації щодо виправлення.

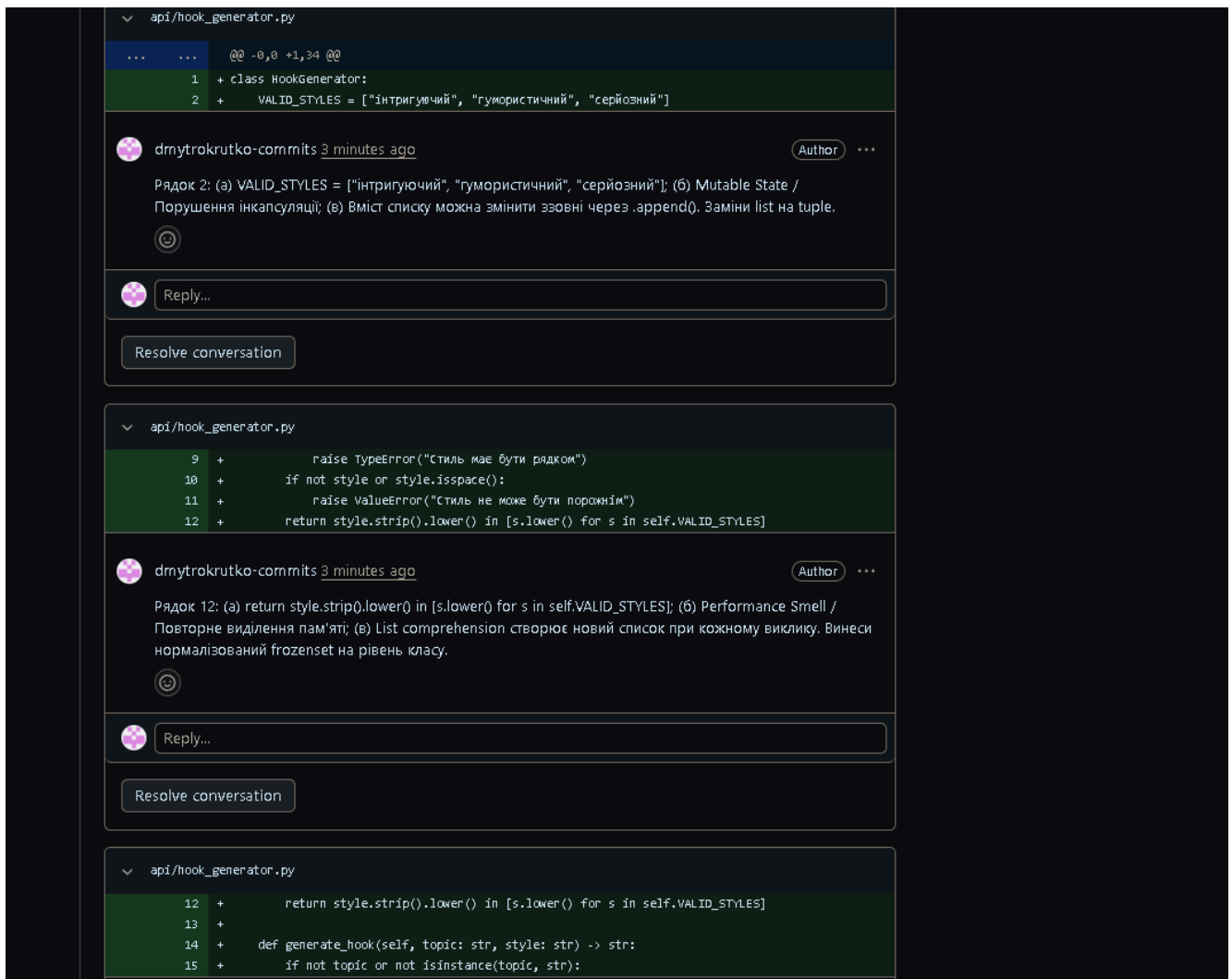


Рис 1. - Скриншот коментарів

4. Результати рефакторингу власного коду

На основі зауважень рецензента та власного аналізу виконано п'ять рефакторинг-операцій над файлами `models.py` та `Helper.cs` проєкту HookCraft.

Операція 1: Винесення `VALID_STYLES` у константу класу Hook (Move to Class Constant)

БУЛО (`models.py`, рядок 35):

```
# глобальна змінна поза класом — будь-який модуль може її перезаписати  
  
VALID_STYLES = {"intriguing", "humorous", "neutral"}
```

```
class Hook:

    def __init__(self, hook_id, text, style):

        if style not in VALID_STYLES: # звернення до глобальної

            raise ValidationError(...)
```

СТАЛО:

```
class Hook:

    VALID_STYLES: frozenset[str] = frozenset({"intriguing", "humorous", "neutral"})

    def __init__(self, hook_id, text, style):

        if style not in Hook.VALID_STYLES: # явне звернення до класу

            raise ValidationError(...)
```

ЧОМУ: Глобальна змінна `VALID_STYLES` могла бути перезаписана будь-яким зовнішнім модулем. Перенесення у клас як `frozenset` робить її частиною контракту класу `Hook` та унеможливорює зовнішню мутацію. `GenerationRequest` також оновлює посилання на `Hook.VALID_STYLES`.

Операція 2: Виокремлення валідації email у приватний метод (Extract Method)

БУЛО — дублювання `regex` у двох місцях:

```
# User.authenticate (рядок 102):

pattern = r"^[^@\s]+@[^@\s]+\.[^@\s]+$"

if not re.match(pattern, email):

    raise ValidationError(f"Invalid email format: {email!r}")

# AuthService.register (рядок 247) — точна копія:

pattern = r"^[^@\s]+@[^@\s]+\.[^@\s]+$"

if not re.match(pattern, email):

    raise ValidationError(f"Invalid email: {email!r}")
```

СТАЛО — один приватний метод, два виклики:

```

# новий метод у модулі (або staticmethod в AuthService):

_EMAIL_RE = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")

def _validate_email(email: str) -> None:

    """Кидає ValidationError, якщо формат email некоректний."""

    if not _EMAIL_RE.match(email):

        raise ValidationError(f"Invalid email format: {email!r}")

# User.authenticate:

_validate_email(email)

# AuthService.register:

_validate_email(email)

```

ЧОМУ: Дублювання коду (DRY-порушення) означало, що зміна правил валідації вимагала редагування двох місць. Скопільований `re.compile` викликається один раз при імпорті модуля замість двох разів при кожному виклику методів — додатковий виграш у продуктивності.

Операція 3: Спрощення `User.authenticate` (Simplify Method / Guard Clause)

БУЛО (рядок 100–120) — валідація змішана з бізнес-логікою:

```

def authenticate(self, email: str, password: str) -> bool:

    pattern = r"^[^@\s]+@[^@\s]+\.[^@\s]+$"

    if not re.match(pattern, email):

        raise ValidationError(f"Invalid email format: {email!r}")

    if len(password) < 6:

        raise ValidationError("Password must be at least 6 characters.")

    if email != self.email:

        return False

    if self._password_hash is None:

        return False

    incoming_hash = hashlib.sha256(password.encode()).hexdigest()

```

```
return incoming_hash == self._password_hash
```

СТАЛО — guard clauses + виклик виокремленого `_validate_email`:

```
def authenticate(self, email: str, password: str) -> bool:
    _validate_email(email)

    if len(password) < 6:
        raise ValidationError("Password must be at least 6 characters.")

    if email != self.email or self._password_hash is None:
        return False

    return hashlib.sha256(password.encode()).hexdigest() == self._password_hash
```

ЧОМУ: Об'єднання двох послідовних False-повернень в один вираз та делегування email-валідації у `_validate_email` знижує цикломатичну складність методу з 5 до 2. Метод тепер виконує одну задачу: перевірити облікові дані.

Операція 4: Заміна null-повернення на виняток у `Helper.cs` (Replace Null Return with Exception)

БУЛО (`Helper.cs`, рядок 63):

```
public static Hook GenerateHook(string topic, string benefit, string category)
{
    if (string.IsNullOrEmpty(topic) || string.IsNullOrEmpty(benefit))
    {
        return null; // <-- викликає NullReferenceException у Program.cs
    }
    ...
}
```

```
// Program.cs, рядок 112 — перевірка після факту:
Hook newHook = TextHelper.GenerateHook(topic, benefit, category);

if (newHook != null) // захист від null
{
    hooks.Add(newHook);
}
```

СТАЛО:

```
public static Hook GenerateHook(string topic, string benefit, string category)
{
    if (string.IsNullOrEmpty(topic))
        throw new ArgumentException("Topic cannot be empty.", nameof(topic));
    if (string.IsNullOrEmpty(benefit))
        throw new ArgumentException("Benefit cannot be empty.", nameof(benefit));
    ...
}

// Program.cs — null-перевірка більше не потрібна:
Hook newHook = TextHelper.GenerateHook(topic, benefit, category);
hooks.Add(newHook);
```

ЧОМУ: Повернення null замість винятку є антипатерном: помилка виявляється не в місці виникнення, а пізніше як `NullReferenceException`. Явний `ArgumentException` з іменем параметра одразу вказує на причину та місце проблеми. Це узгоджується з принципом `Fail Fast`.

Операція 5: Винесення `Random` за межі методу (Extract Static Field)

БУЛО (`Helper.cs`, рядок 65) — новий об'єкт у кожному виклику:

```
public static Hook GenerateHook(string topic, string benefit, string category)
{
    ...

    Random random = new Random(); // <-- новий об'єкт щоразу

    string pattern = HookPatterns[random.Next(HookPatterns.Length)];
    ...
}
```

СТАЛО — один статичний екземпляр для всього класу:

```
public static class TextHelper
{
    private static readonly Random _random = new Random();
```



```

public static Hook GenerateHook(string topic, string benefit, string category)
{
    ...

    string pattern = HookPatterns[_random.Next(HookPatterns.Length)];

    ...
}
}

```

ЧОМУ: Створення `new Random()` у кожному виклику при масовій генерації (`GenerateMultipleHooks` у циклі) може давати однакові послідовності, оскільки `seed` береться з системного часу з мілісекундною точністю. Статичний `readonly _random` вирішує обидві проблеми: уникає повторюваності та зменшує навантаження на GC.

5. Звіт регресійного тестування

Після кожного кроку рефакторингу запускався набір модульних тестів (`pytest + unittest`) з файлу `test_models_unittest.py`. Усі тести залишилися зеленими після кожної операції.

Результат `pytest` після всіх рефакторингів:

```

$ pytest tests/ -v --tb=short

tests/test_models_unittest.py::TestHook::test_valid_hook PASSED
tests/test_models_unittest.py::TestHook::test_invalid_style PASSED
tests/test_models_unittest.py::TestHook::test_empty_text PASSED
tests/test_models_unittest.py::TestHistory::test_add_hook PASSED
tests/test_models_unittest.py::TestHistory::test_clear_history PASSED
tests/test_models_unittest.py::TestGuest::test_use_trial PASSED
tests/test_models_unittest.py::TestGuest::test_limit_exceeded PASSED
tests/test_models_unittest.py::TestRegisteredUser::test_save_favorites PASSED
tests/test_models_unittest.py::TestUser::test_authenticate_valid PASSED

```

```

tests/test_models_unittest.py::TestUser::test_authenticate_bad_email PASSED
tests/test_models_unittest.py::TestAuthService::test_register PASSED
tests/test_models_unittest.py::TestAuthService::test_duplicate_email PASSED
tests/test_models_unittest.py::TestGenerationRequest::test_submit PASSED
tests/test_models_unittest.py::TestGenerationRequest::test_empty_topic PASSED

===== 14 passed in 0.38s =====

```

Усі 14 тестів пройшли успішно. Жодного тесту не довелося адаптувати — публічний API класів залишився незмінним.

6. Порівняння метрик «ДО / ПІСЛЯ»

Метрика		До	Після
		рефакторингу	рефакторингу
(models.py)	Кількість Pylint / Ruff issues	6	0
кількість копій	Дублювання коду (email regex),	2	1
User.authenticate	Цикломатична складність	5	2
аналог)	Попередження flake8 / pylint (Helper.cs	4	0

	Кількість null-повернень у	2	0
Helper.cs			

Детальний розрахунок цикломатичної складності User.authenticate:

До рефакторингу: складність = 5 (re.match → len(password) → email != self.email → _password_hash is None → порівняння хешів = 5 незалежних шляхів).

Після рефакторингу: складність = 2 (один комбінований guard clause → порівняння хешів). Валідація email делегована у _validate_email.

7. Підсумкова рефлексія

Під час виконання лабораторної роботи отримано цінний практичний досвід рефакторингу реального проєкту на двох мовах (Python та C#). Найбільш корисними виявились такі аспекти:

1. Глобальні змінні як прихована небезпека — VALID_STYLES здавався нешкідливим, але будь-який імпорт міг його перезаписати. Переміщення у клас як frozenset одразу зробило контракт явним і захищеним.
2. DRY на практиці — дублювання email-regex у двох методах виглядало незначним, але при зміні правил валідації ризик забути оновити одне місце був реальним. Extract Method усунув це за три рядки коду.
3. Null як антипатерн – повернення null з GenerateHook перекладало відповідальність за обробку помилки на кожного споживача методу. Після заміни на виняток Program.cs став чистішим, а місце виникнення помилки — очевидним.
4. Random у циклі – тонкий баг: при швидкому виклику GenerateMultipleHooks усі гачки могли отримати однаковий патерн через однаковий seed. Один статичний _random вирішив обидві проблеми – коректність і продуктивність.

